

Modül 20: Paketler ve Modüller

Paketler adları düzenler. Sınıf yolu JVM'e nereye bakacağını söyler. Modüller ikisinin üstüne zorlama ekler.

Burada ihtiyacınız olanın çoğu pratik: bir build dosyasını okumaya, bir başlangıç hatasını teşhis etmeye ve

`--add-opens` seçeneğinin neden var olduğunu anlamaya yetecek kadar.

Neler Öğreneceksiniz

- Paketten dizine kural, ve gerçek bir erişim seviyesi olarak paket-özel
- Sınıf yolu, ve aynı şey olmayan iki başlangıç hatası
- Modül içe aktarma bildirimleri, ve ne zaman kullanılmayacağı
- Bir jar kurmak, ve bir modülün jar'ın yapamadığı neyi eklediği

Başka Bir Dilden Geliyorsanız

	Python	Java
Ad alanı	modül = dosya, paket = izin	dosyada bildirilen paket
Düzen zorunluluğu	import yolu ile	derleyici tarafından
Arama yolu	<code>sys.path</code>	sınıf yolu ya da modül yolu
Her şeyi içe aktarma	<code>from x import *</code>	<code>import x.*</code> ya da <code>import module m</code>
Dosyalar arası gizlilik	alt çizgi geleneği	paket-özel, zorlanan
Dağıtım birimi	wheel	jar

İnsanları yakalayan satır zorunlu düzen. Java'da `package` ifadesi ile izin yolu eşleşmek **zorundadır**, ve derleyici bunu size söyler.

Ders

Paketler

```
package com.example.util;    ->    com/example/util/Greeter.java
```

Derleyici dizini, kendisine verilmeyen kaynakları bulmak için kullanır; JVM de çalışma zamanında sınıfları bulmak için.

`Packages.java` geçici bir dizinde küçük iki paketli bir proje kurar, gerçek `javac` ile derler ve çalıştırır:

```
javac exit code: 0
produced: [classes/com/example/app/Main.class, classes/com/example/util/Greeter.class]
<< Hello, world >>
```

Adlandırma: ters alan adı, tamamı küçük harf. Ters çevirme, iki kuruluşun çakışmaması için var.

Gerçek kod için asla varsayılan paketi kullanmayın. `package` ifadesi olmayan bir sınıf, paketi olan hiçbir sınıf tarafından içe aktarılamaz, dolayısıyla düzenli hiçbir yerden kullanılamaz.

Paket-özel gerçek bir erişim seviyesidir, ve insanların unuttuğu olanıdır:

Seviye	Erişim
<code>private</code>	yalnızca bu sınıf
<i>(hiçbiri)</i>	bu paket
<code>protected</code>	bu paket artı her yerdeki alt sınıflar
<code>public</code>	herkes

İşbirliği yapan sınıfların, bir şeyi dünyaya açmadan paylaşma biçimi budur.

Sınıf yolu

Dizin ve jar dosyalarından oluşan, **sırayla aranan** bir liste:

```
java -cp classes:lib/one.jar:lib/two.jar com.example.Main
java -cp "classes;lib/*" com.example.Main      (Windows ; kullanır)
```

İki hata aynı şey değildir. `TheClasspath.java` dosyasından:

ClassNotFoundException denetlenen bir istisnadır. Bir şey çalışma zamanında bir sınıfı *ismen* istedi ve bulunamadı. Genellikle yansıma, bir JDBC sürücüsü ya da yapılandırılmış bir sınıfı yükleyen bir çatı.

NoClassDefFoundError bir Error'dur. Sınıf derleme zamanında mevcuttu ve şimdi eksik ya da kullanılamaz durumda. Ve insanların yanlış teşhis ettiği durum şu:

```
first attempt: ExceptionInInitializerError, caused by java.lang.IllegalStateException:
the static initialiser failed
second attempt: NoClassDefFoundError: Could not initialize class TheClasspath$Broken
```

Statik ilklendiricisi fırlatan bir sınıf gerçek sebebi **bir kez** bildirir. Sonraki her deneme hiçbir ipucu olmadan `NoClassDefFoundError` der.

Sınıf yolunda olduğundan emin olduğunuz bir sınıf için `NoClassDefFoundError` görüyorsanız, logda **yukarı** kaydırın. Gerçek başarısızlık daha önce bildirildi ve ilgisiz görünüyordu.

Kimsenin beklemediği başarısızlık biçimi: aynı sınıfı içeren iki jar. Sınıf yolu sırayla aranır, ilki kazanır, ikincisi sessizce yok sayılır. Belirtiler, var olmayan bir metot ya da değiştirdiğinizi sandığınız bir sürümden gelen davranış. Modül sisteminin çözmek için kurulduğu problem budur.

Modül içe aktarma bildirimleri

JEP 511, JDK 25'te kesinleşti:

```
import module java.base;
```

Tek satır, o modülün dışa açtığı her public paketi içe aktarır. `ModuleImports.java` dosyasından, başka hiçbir import olmadan:

```
names:    [ada, grace, alan]
path:     /tmp/example.txt
time:     2026-09-23
```

Bu bir **derleme zamanı kolaylığıdır**. Bytecode birebir aynıdır.

Script'ler, tek dosyalık programlar ve öğretim örnekleri için **kullanın**. Gerçek bir kod tabanında **kullanmayın**: açık import'lar bir dosyanın neye bağlı olduğunu belgeler, böylece inceleyen kişi bir dosyanın yeni bir yere uzanmaya başladığını görebilir. Ayrıca iki modül aynı basit adı dışa açtığında belirsizlik mümkün hâle gelir.

Jar'lar ve modüller

`JarsAndModules.java` dosyasından:

```
jar --create --file tool.jar --main-class com.example.tool.Tool -C classes .
```

```
META-INF/MANIFEST.MF
com/example/tool/Tool.class
```

`MANIFEST.MF`, `--main-class` seçeneğinin yazdığı şeydir, ve `java -jar` komutunun nereden başlayacağını bilmesinin sebebi budur.

Bir **modül** bir bildirim ekler:

```
module com.example.tool {
    requires java.logging;    // what it needs
    exports com.example.tool; // what others may use
}
```

Düz bir jar'ın yapamadığı üç şey:

1. **Güçlü kapsülleme**. Dışa açılmamış bir pakete dışarıdan erişilemez, yansıma ile bile. `public` artık evrensel olarak erişilebilir anlamına gelmez.

2. **Başlangıçta denetlenen bağımlılıklar.** Eksik bir `requires`, üretimde bir saat sonra `NoClassDefFoundError` olarak değil, JVM başlarken hata olarak ortaya çıkar.
3. **Bölünmüş paket yok.** İki modül aynı paketi içeremez, ki bu tam olarak yukarıda anlatılan sessiz gölgelemedir.

Çoğu projenin hâlâ neden sınıf yolu kullandığı: modülerleştirmek, her bağımlılığın da modüler olmasını ya da otomatik modül sayılmasını gerektirir, ki bu size kapsülleme olmadan ad denetimi verir.

Çoğu geliştiricinin modül sistemiyle gerçekte karşılaştığı yer: bir kütüphanenin `java.base` içine yansıma yapmasından kaynaklanan `InaccessibleObjectException`, `--add-opens` ile çözülen. Pratik asgari şart o bayrağın neden var olduğunu bilmek: `java.base` içini size dışa açmıyor, ve `--add-opens` bunu geçersiz kılıyor.

Araçlar

```
jar --list --file x.jar           # içine bakın
jar --describe-module --file x.jar # modüler mi
jdeps --list-deps x.jar          # neye ihtiyacı var
javadoc -d docs src/**/*.java   # dokümantasyon üretin
```

Çalıştırın

```
java modules/20-packages-and-modules/examples/Packages.java
java modules/20-packages-and-modules/examples/TheClasspath.java
java modules/20-packages-and-modules/examples/ModuleImports.java
java modules/20-packages-and-modules/examples/JarsAndModules.java

./scripts/verify-examples.sh modules/20-packages-and-modules
```

Bunlardan ikisi geçici bir dizinde gerçek bir paketli proje kurar, `javac` ile derler ve sonra siler. Geride hiçbir şey kalmaz.

Sık Yapılan Hatalar

Paket ifadesinin dizinle eşleşmemesi. Derleyici bunu söylüyor; onunla kavga etmek yerine inanın.

Varsayılan paketi kullanmak. Paketi olan hiçbir şey sizi içe aktaramaz.

Refleksle her şeyi `public` yapmak. Paket-özel var ve işbirliği yapan sınıflar için genellikle doğru olan o.

`NoClassDefFoundError` hatasını **eksik jar diye teşhis etmek**, oysa daha önce bir statik ilklendirici başarısız olmuş.

Sınıf yolunda bir kütüphanenin iki sürümü. İlki kazanır, sessizce.

Üretim kodunda `import module` . Bir dosyanın neye bağlı olduğunu gizler.

`--add-opens` seçeneğini anlamadan eklemek. Başka bir modülün kapsüllemesinde açılmış bir deliktir. Bazen gerekli, asla bedava.

Özet Çıkarımlar

- **Paket ifadesi ile dizin eşleşmelidir**, ve derleyici bunu zorlar.
- **Paket-özel varsayılan erişim seviyesidir**, bir seviyenin yokluğu değil.
- **Sınıf yolu sırayla aranır**, dolayısıyla tekrarlanan bir sınıf sessizce gölgelenir.
- `ClassNotFoundException` ismen istendi demektir; `NoClassDefFoundError` derleme zamanında oradaydı demektir. Başarısız bir statik ilklendirici ikincisine sebep olur ve birincisini gizler.
- `import module` script'ler için bir derleme zamanı kolaylığıdır, kod tabanları için değil.
- **Modüller kapsülleme, başlangıç zamanında bağımlılık denetimi ve bölünmüş paket yasağı** ekler, ki `--add-opens` bu yüzden var.

Ödev

[homework/README.md](#)

Sıfırdan iki paketli bir proje kurun, jar'layın, sonra üç başlangıç hatasını teşhis edin. Referans çözüm [solutions/20-packages-and-modules/](#) altında.